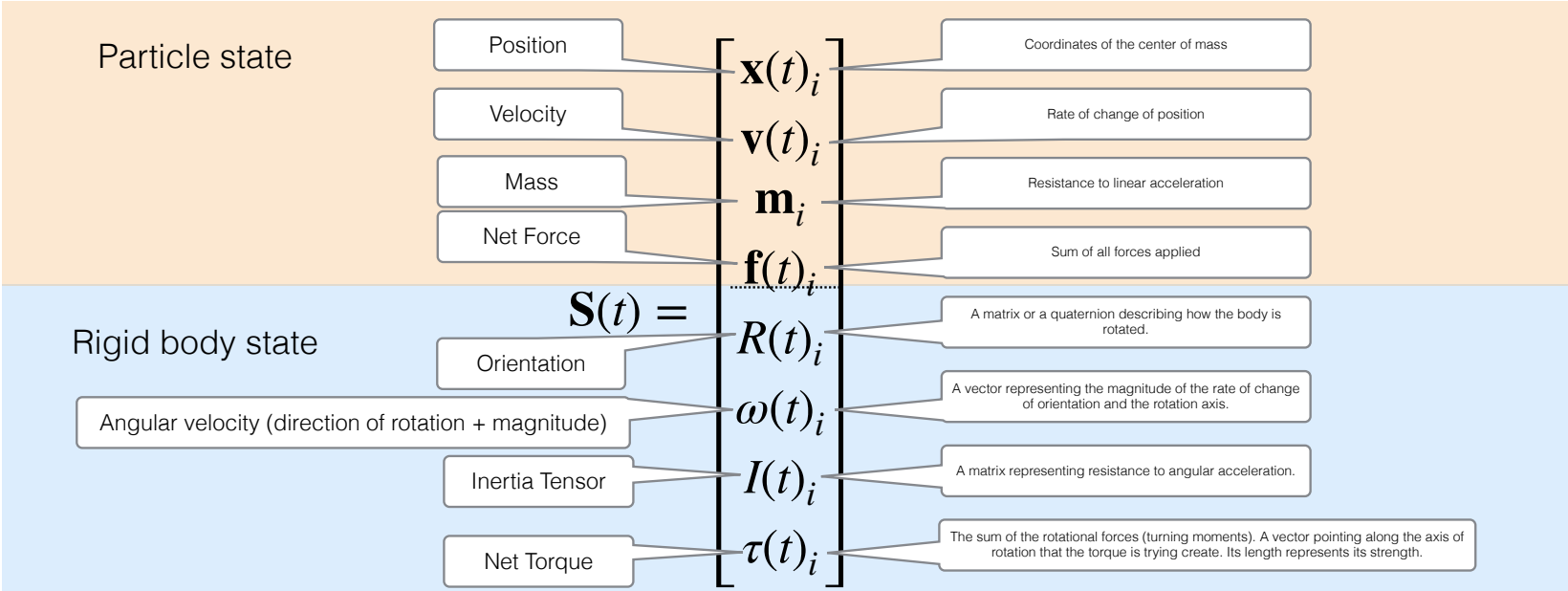
Rigid Body Dynamics

Fernando Birra
Rui Nóbrega

What is a Rigid-Body?

- Definition: A collection of particles where the distances between any two points is constant (no deformation)

- The state now includes orientation ($\mathbf{q}$ or $\mathbf{R}$) and Angular Velocity (ω).



Center of Mass

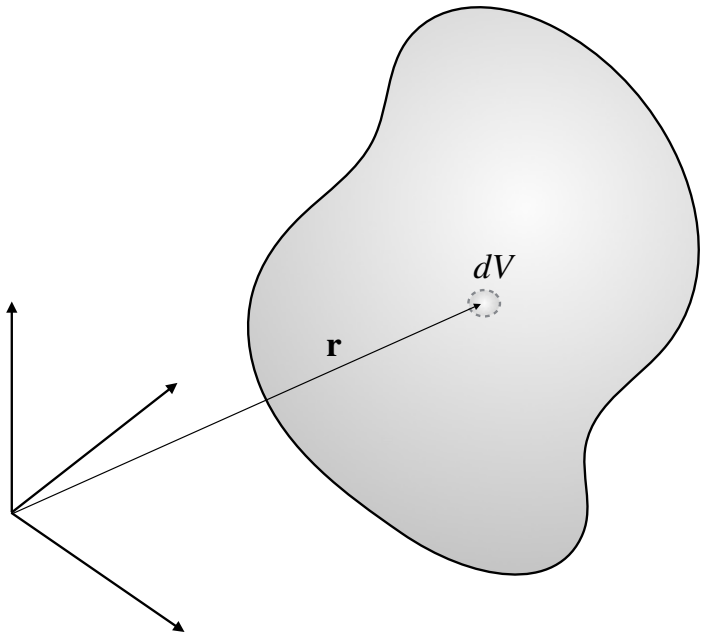
- Center of Mass: point where all the mass could be concentrated for the computation of the first moment: $m\mathbf{r}$ (mass x distance)

- Continuous Formula:

$$\mathbf{r}_{CM} = \frac{1}{m}\int\int\int_V \mathbf{r}\rho(\mathbf{r})dV$$
$$m = \int\int\int_V \rho(\mathbf{r})dV$$

- Discrete Approximation:

$$\mathbf{r}_{CM} = \frac{\sum_{i=1}^{n} m_i\mathbf{r}_i}{\sum_{i=1}^{n} m_i} \Leftrightarrow \sum_{i=1}^{n} m_i\mathbf{r}_{CM} = \sum_{i=1}^{n} m_i\mathbf{r}_i$$



$\rho(\mathbf{r})$: mass density at location $\mathbf{r}$

dV: differential volume

Orientation

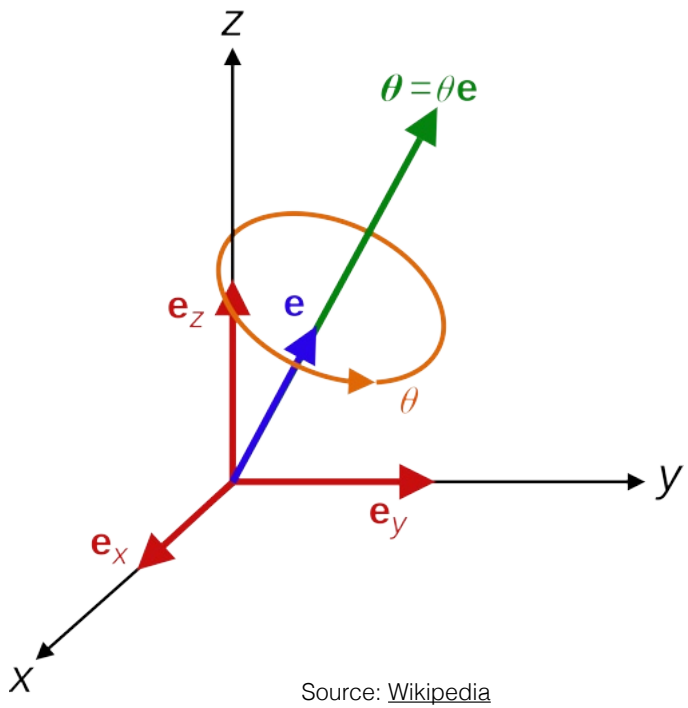
- The orientation of a rigid body can be described by some arbitrary rotation matrix:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

- This orthogonal matrix rotates the original axes to their current position.

- It can also be described as a rotation quaternion $\mathbf{q}$:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) & e_x \sin(\theta/2) & e_y \sin(\theta/2) & e_z \sin(\theta/2) \end{bmatrix}$$



Source: Wikipedia

Orientation

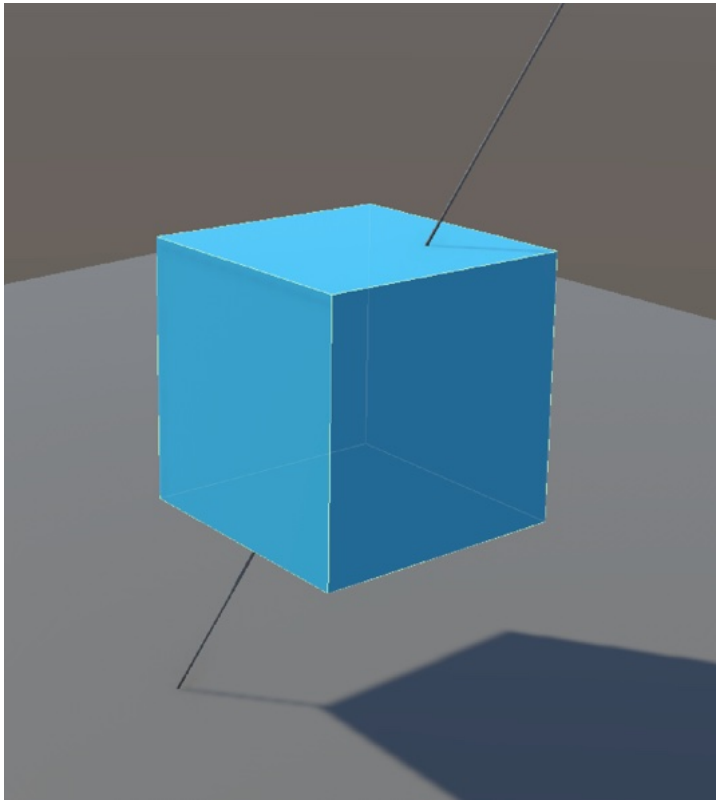
- The orientation of a rigid body can be described by some arbitrary rotation matrix:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

- This orthogonal matrix rotates the original axes to their current position.

- It can also be described as a rotation quaternion $\mathbf{q}$:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) & e_x \sin(\theta/2) & e_y \sin(\theta/2) & e_z \sin(\theta/2) \end{bmatrix}$$



Orientation

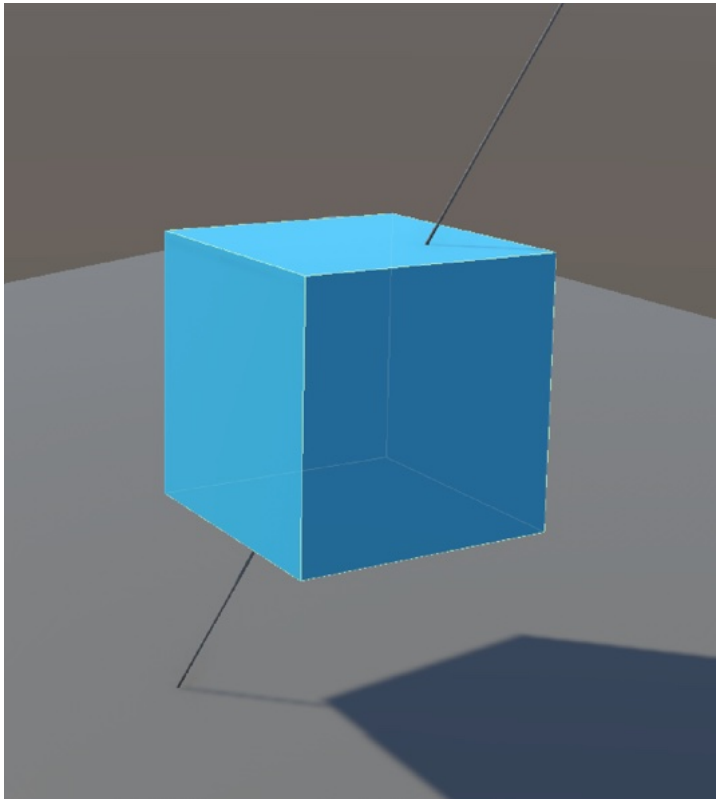
- The orientation of a rigid body can be described by some arbitrary rotation matrix:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

- This orthogonal matrix rotates the original axes to their current position.

- It can also be described as a rotation quaternion $\mathbf{q}$:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) & e_x \sin(\theta/2) & e_y \sin(\theta/2) & e_z \sin(\theta/2) \end{bmatrix}$$



Orientation

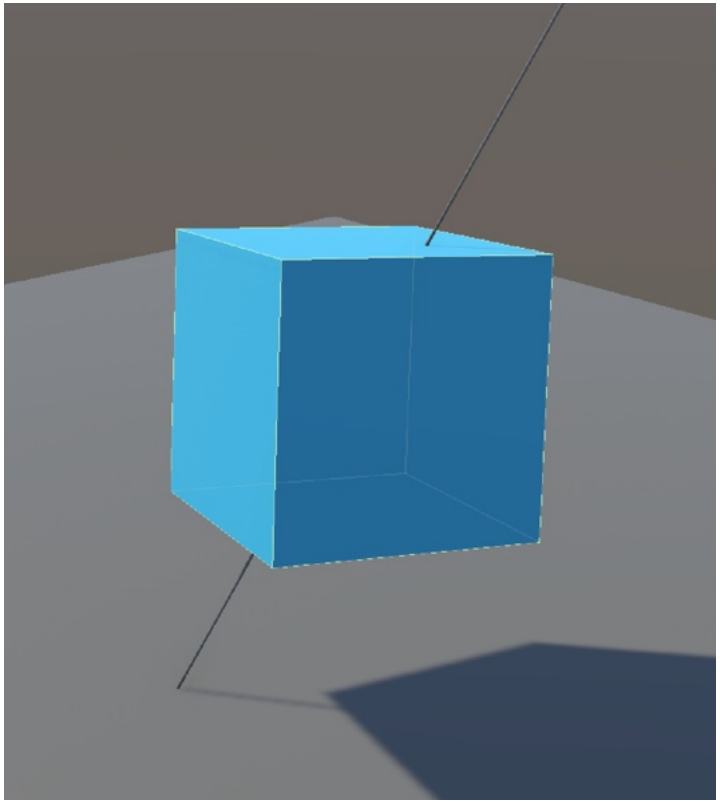
- The orientation of a rigid body can be described by some arbitrary rotation matrix:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

- This orthogonal matrix rotates the original axes to their current position.

- It can also be described as a rotation quaternion $\mathbf{q}$:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) & e_x \sin(\theta/2) & e_y \sin(\theta/2) & e_z \sin(\theta/2) \end{bmatrix}$$



Orientation

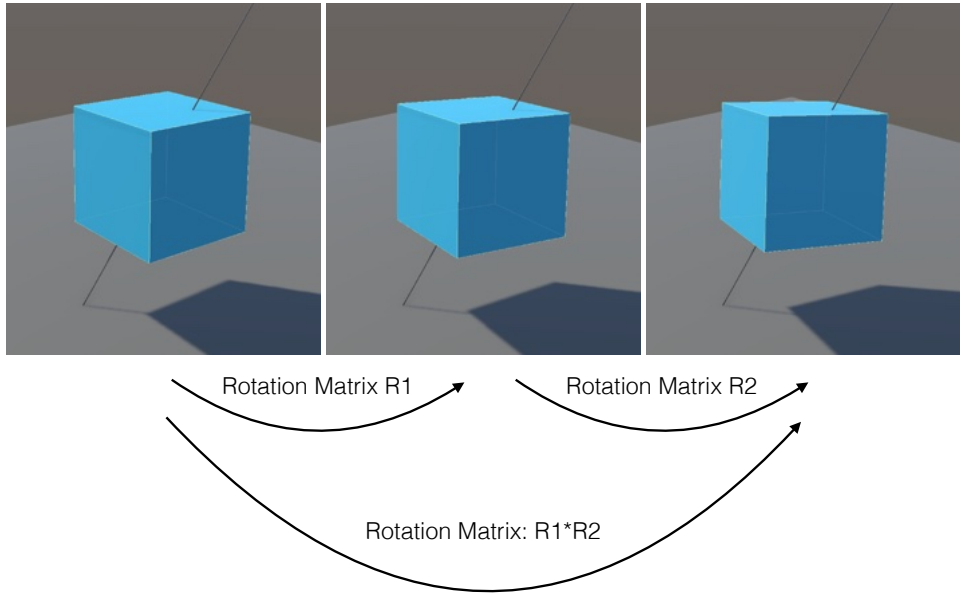
- The orientation of a rigid body can be described by some arbitrary rotation matrix:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

- This orthogonal matrix rotates the original axes to their current position.

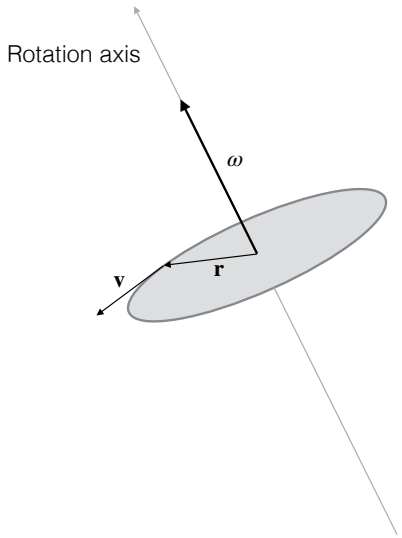
- It can also be described as a rotation quaternion $\mathbf{q}$:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) & e_x \sin(\theta/2) & e_y \sin(\theta/2) & e_z \sin(\theta/2) \end{bmatrix}$$



Angular Velocity

- Describes the change in orientation of a body
- Angular velocity is represented by a vector encoding both the axis of rotation and its rotational speed (magnitude)



Relation between linear and angular velocity:

$$\mathbf{v} = \omega \times \mathbf{r}$$

Angular velocity:

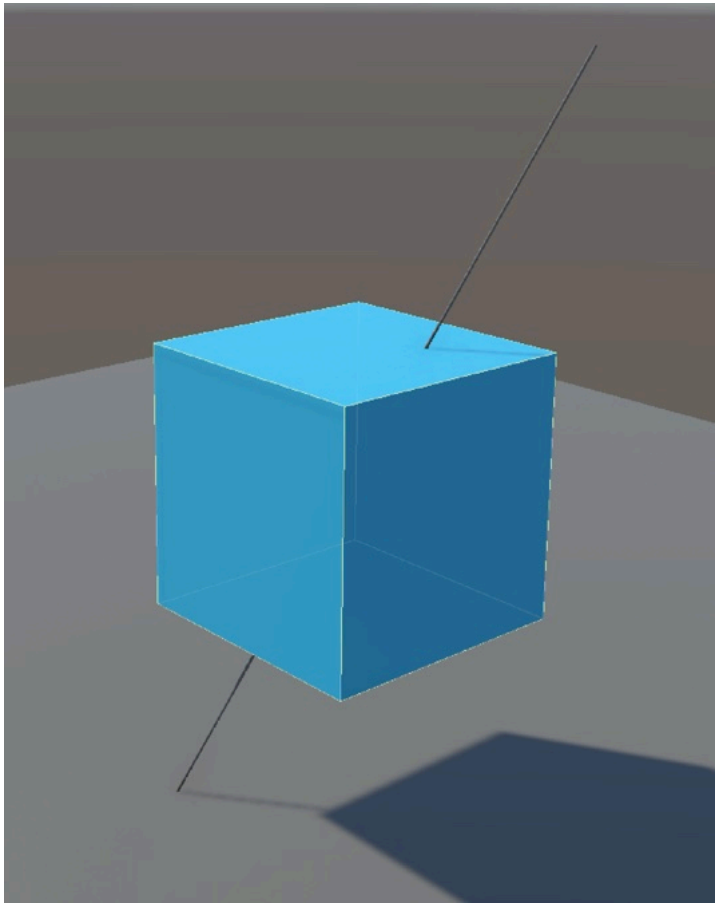
$$\omega = \frac{\mathbf{r} \times \mathbf{v}}{\mathbf{r} \cdot \mathbf{r}}$$

Angular speed

$$\|\omega\| = d\theta/dt$$

Angular Velocity

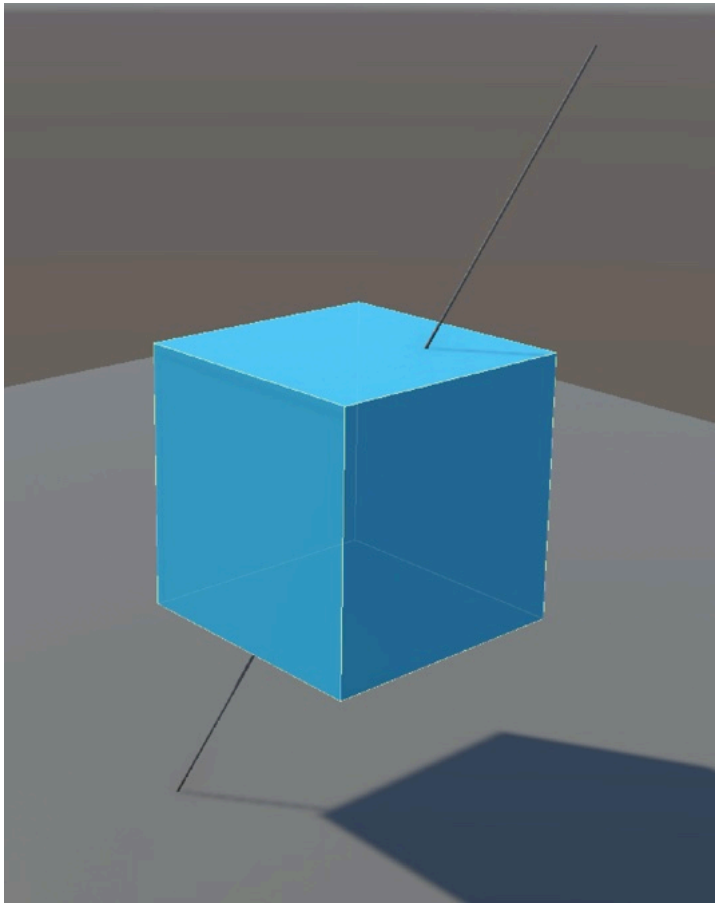
Speed=10



- Describes the change in orientation of a body
- Angular velocity is represented by a vector encoding both the axis of rotation and its rotational speed (magnitude)

Angular Velocity

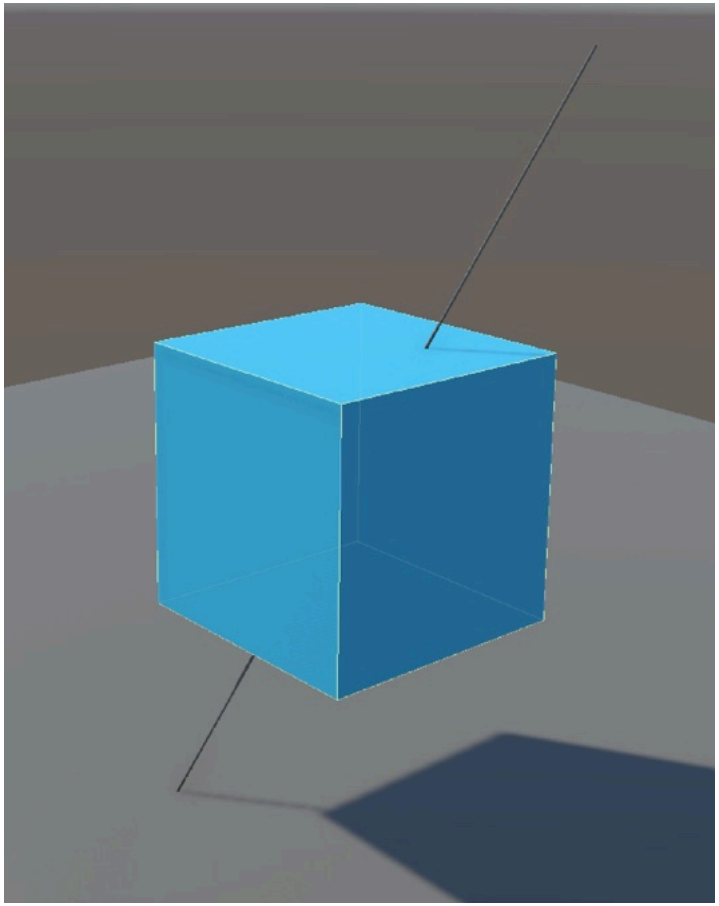
Speed=20



- Describes the change in orientation of a body
- Angular velocity is represented by a vector encoding both the axis of rotation and its rotational speed (magnitude)

Angular Velocity

Speed=50



- Describes the change in orientation of a body
- Angular velocity is represented by a vector encoding both the axis of rotation and its rotational speed (magnitude)

Inertia Tensor

- For a set of particles, the Inertia Tensor about a specific axis is given by:

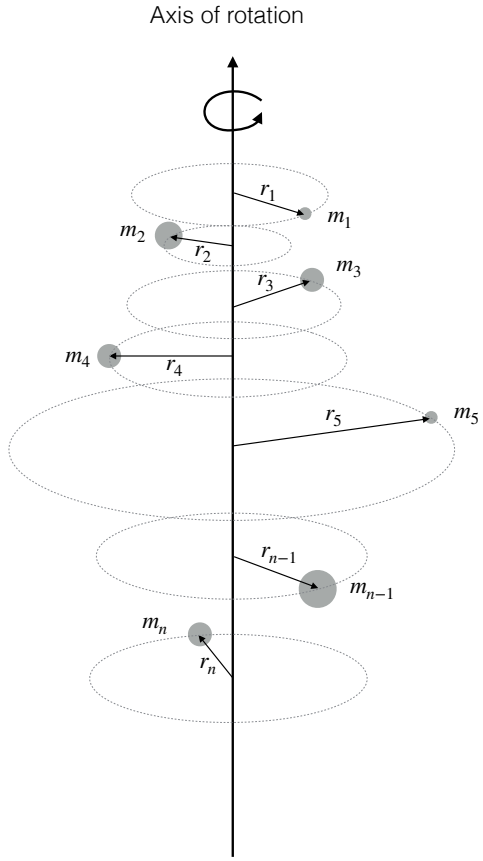
$$I = \sum_{i=1}^{n} m_i r_i^2$$

- For a solid body (continuous):

$$I = \int r^2 dm$$

- If the material has constant density:

$$I = \int_V \rho r^2 dV$$

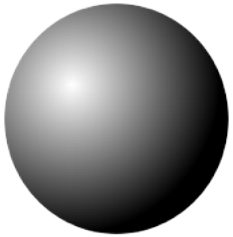


Inertia Tensor

- Inertia Tensor (I): A 3×3 matrix describing how mass is distributed around the axes. Resistance to rotational acceleration.

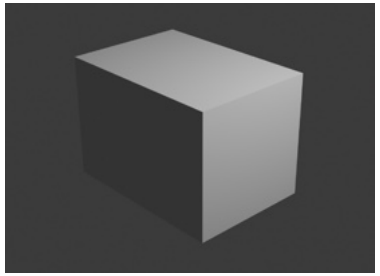
$$I = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}$$

- Engines use pre-derived formulas for basic shapes (boxes, spheres, cylinders) computed around their center of mass.



$$I_{xx} = I_{yy} = I_{zz} = \frac{2}{5}mr^2$$

Solid spere: radius r



$$I_{xx} = \frac{m}{12}(h^2 + d^2)$$
$$I_{yy} = \frac{m}{12}(w^2 + d^2)$$
$$I_{zz} = \frac{m}{12}(w^2 + h^2)$$

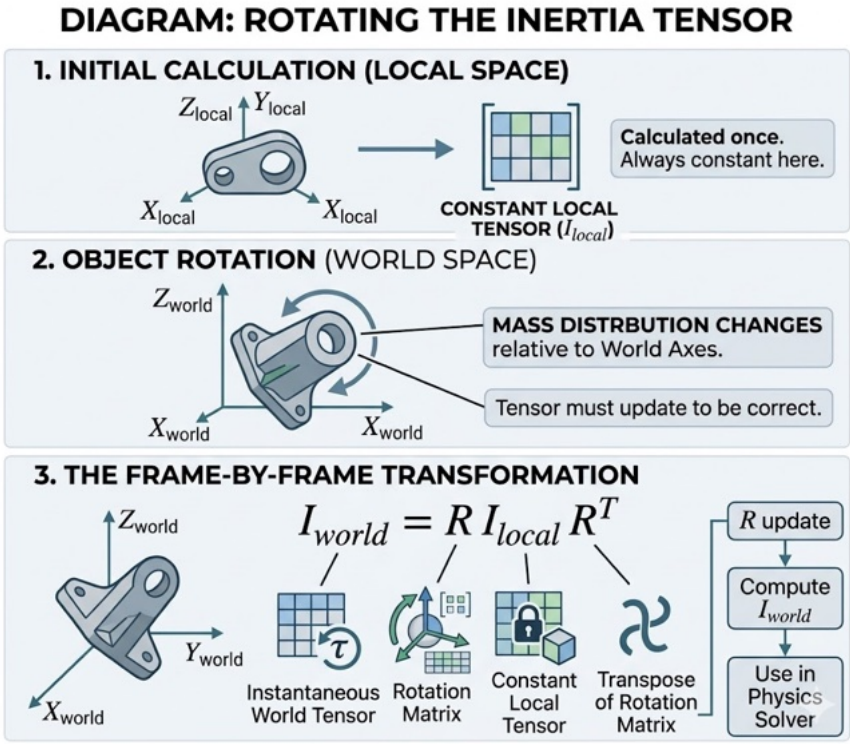
Solid Box: $w \times h \times d$

Local vs World Inertia Tensors

- The Inertia Tensor is usually calculated once, in object's local space.*
- As the object spins in the world, its mass distribution relative to the world axes changes.
- To keep it correct, the tensor must rotate into world space **every frame**.

$$I_{world} = RI_{local}R^T$$

where R is the rotation matrix from local to world coordinates.



*If the chosen local coordinates don't give a diagonal inertia tensor, compute Eigen vectors of inertia tensor to discover the principal axis of the object. In that reference system the Inertia Tensor is always a diagonal matrix.

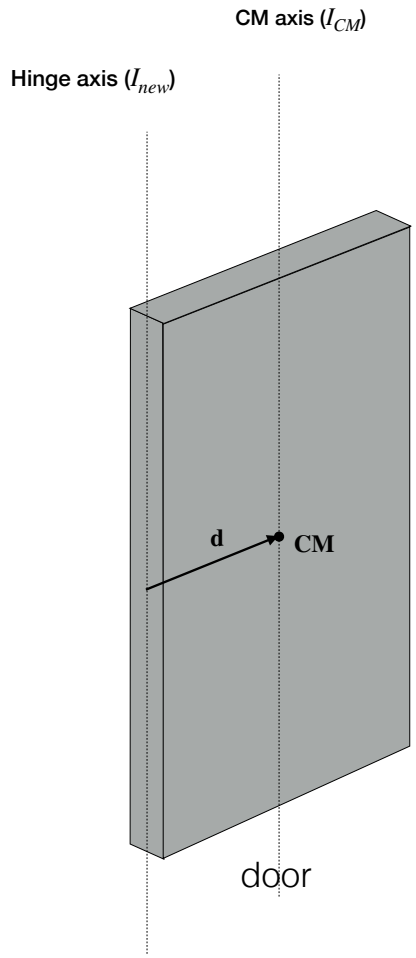
Inertia Tensor: Parallel Axis Theorem

- If you need the inertia tensor around a point other than the center of mass (like a hinge on a door):

$$I_{new} = I_{CM} + m([\mathbf{d}]_{\times}{}^{2})$$

Where $\mathbf{d}$ is the displacement vector and $[\mathbf{d}]_{\times}$ is the skew symmetric matrix:

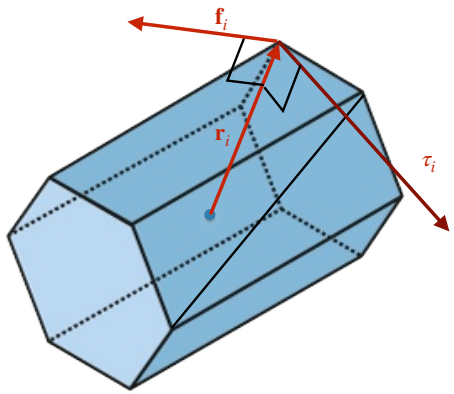
$$[\mathbf{d}]_{\times} = \begin{bmatrix} 0 & -d_z & dy \\ d_z & 0 & -d_x \\ -d_y & d_x & 0 \end{bmatrix}$$



Torque

- Each j independent force acting on body i: $\mathbf{f}_j$
- Net torque on body: $\tau = \sum_j \mathbf{r}_j \times \mathbf{f}_j$
- The relation between torque τ and angular acceleration α is approximately given by:

$$\tau = I\alpha$$



Linear vs Angular

- Position $\mathbf{x}$
- Velocity $\mathbf{v}$
- Acceleration $\mathbf{a}$
- Mass m
- Force $\mathbf{f}$
- Momentum $\mathbf{p} = m\mathbf{v}$
- Newton's 2nd law $\mathbf{f} = m\mathbf{a}$

- Orientation $\mathbf{R}$ (matrix) or $\mathbf{q}$ (quaternion)
- Angular Velocity ω
- Angular Acceleration α
- Inertia Tensor I
- Torque τ
- Angular Momentum $L = I\omega$
- Newton's 2nd law $\tau = I\alpha$

Rigid Body Euler Integration Step

- Update linear state as for particles
- Update angular state:
  - Compute Inertia Tensor in World Coordinates: $I_{world} = RI_{local}R^T$
  - Compute Angular Acceleration (solve for α): $\alpha = I_{world}^{-1}\tau$
  - Integrate Angular Acceleration: $\omega_{i+1} = \omega_i + \alpha \cdot \Delta t$
  - Update Orientation: $\mathbf{R}_{i+1} = \mathbf{R}_i + \dot{\mathbf{R}}_i \Delta t$ ($\dot{\mathbf{R}}_i$ approximates the rate of change of rotation, compute as $[\omega]_\times \mathbf{R}_i$)

    (Due to errors introduced by Euler steps, rotation matrix will need to be re-orthogonalized periodically using, for instance, Gram-Schmidt Orthogonalization)

Engine Rigid Body Tricks

- Sleep State: If a body hans't moved significantly for a while, "put it to sleep" until something hits it.
- Mark rigid bodies objects as **static** (unmovable), **kinematic** (moved by script, instead of forces), or **dynamic** (fully physical)
- Sometimes we only want to know if a collision has occurred for a given object. Mark it as **isTrigger** to avoid computing a response with a physical bounce. A callback will be called instead.

Collision Detection and Response

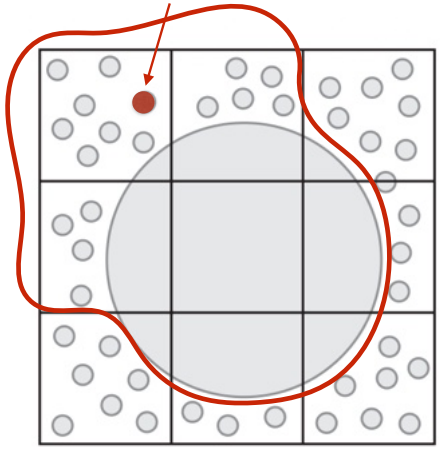
Fernando Birra
Rui Nóbrega

The Collision Detection Pipeline

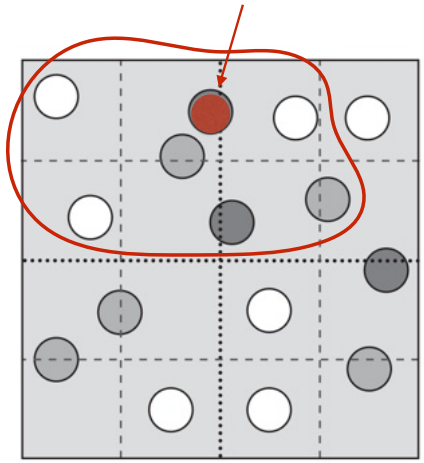
- Computing possible collisions for all rigid bodies quickly becomes an overly complex process:
  - $\mathcal{O}(n^2)$ complexity if we treat all possible pairs
  - Discrete time steps if treated as snapshots may miss collisions
  - Exact contact determination between two moving bodies is expensive
- Strategy: Break process in a pipeline:

Spatial Partitioning

- Before looking at individual shapes we look around to try to find neighbors. Without this we need to perform n^2 checks for n objects.
- Divide the world into a Grid, an Quadtree (2D)/Octree (3D) or a Spatial Hashing. Each object is registered to the “cells” it occupies.
- We only need to check an object with the objects from its neighboring cells.



Regular grid



Quadtree

Broad Phase: Bounding Box Overlap

- For the neighbors from the spatial data structure, we use AABB (axis aligned bounding boxes) or Bounding Spheres.

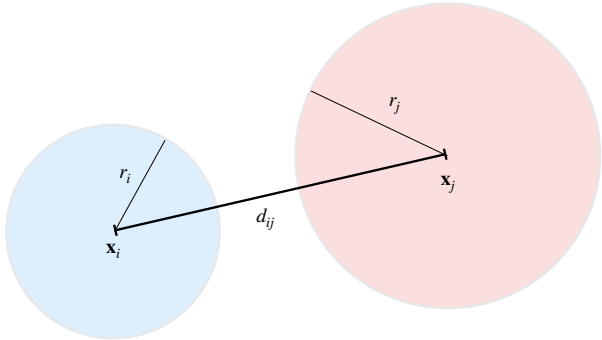
- Two neighbor objects may still be far apart and the Bounding Volumes provide simpler checks that can discard candidates:

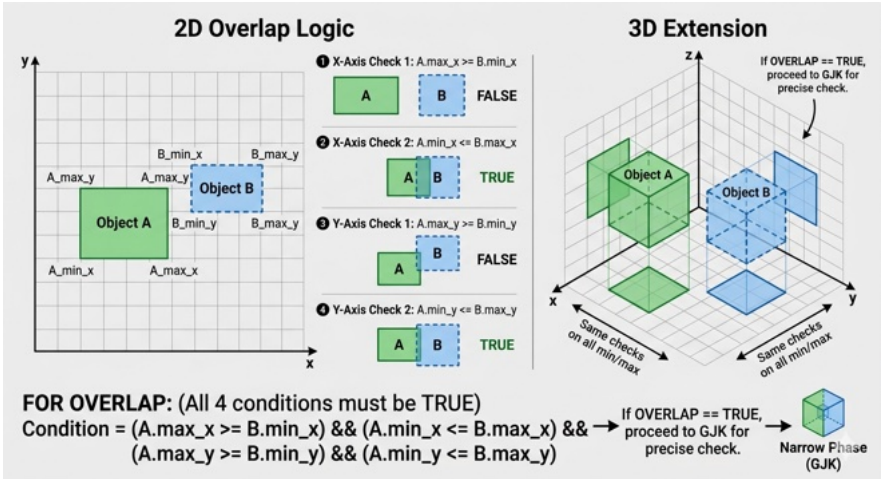
  - Bounding Spheres: $d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\| > r_i + r_j$

  - AABBs: $(x_{max_i} < x_{min_j}) \vee (x_{max_j} < x_{min_i}) \ldots$

- Way faster than do the exact collision tests…

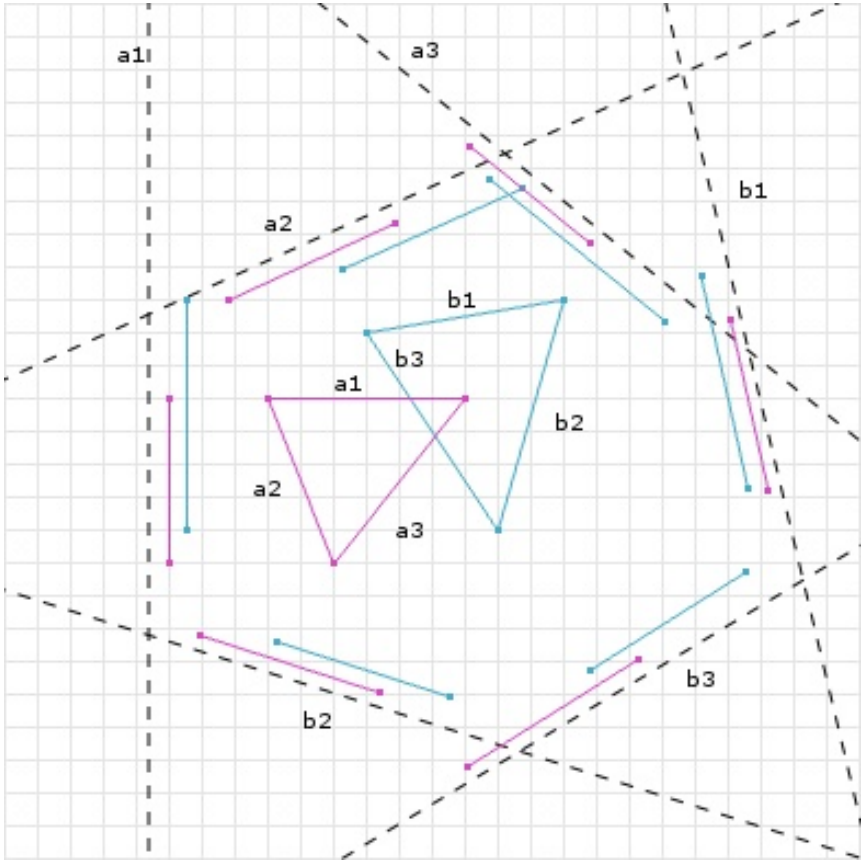
- **Note**: Bounding Volume Hierarchies (**BVH**) are used they can act as both the spatial partition and the broad phase. But they can also be used as a middle phase for very complex triangle meshes.





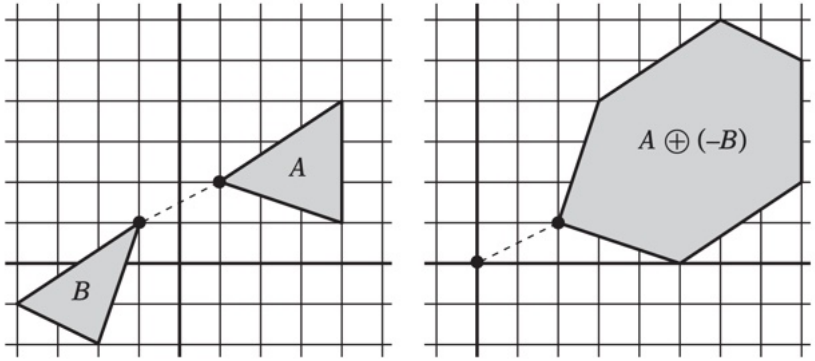
Narrow Phase: Exact Computations (SAT)

- If the bounding boxes overlap apply **GJK** or **SAT**!
- The Separating Axis Theorem (**SAT**) - If you can find a line (an axis) where the shadows (projections) of two convex shapes do not overlap then these shapes are not touching.
- No need to check infinite axis
  - Face normals of Object A
  - Face normals of Object B
  - (In 3D) The cross product of every edge from A with every edge from B.
- If all axes show an overlap, the shapes collide. The axis with the smallest overlap is usually chosen as the **Collision Normal**, and the overlap amount is the **Penetration Depth**.



Narrow Phase: Exact Computations (GJK)

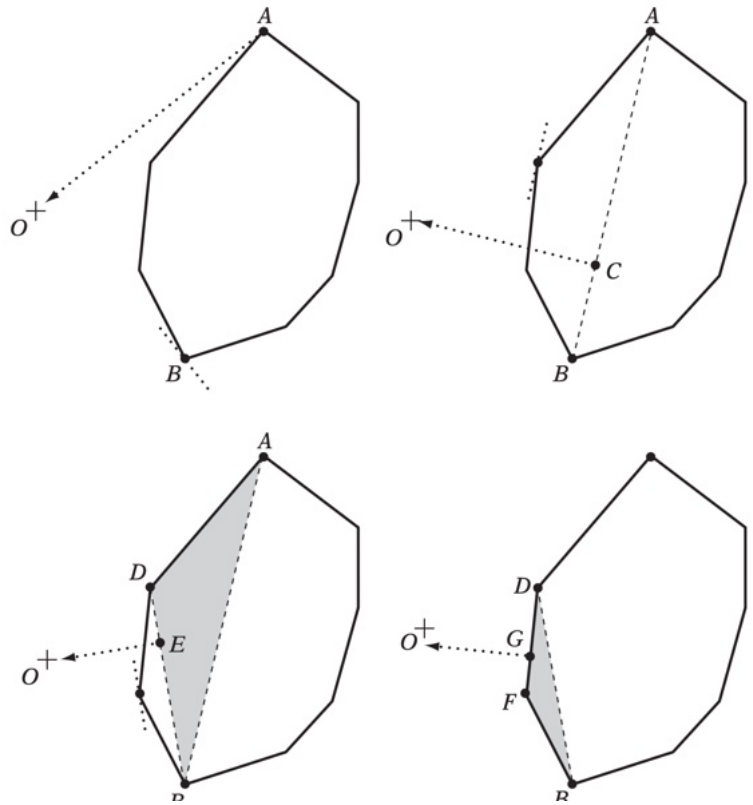
- If the bounding boxes overlap apply **GJK** or **SAT**!
- The **GJK** Algorithm (Gilbert-Johnson-Keerthi) performs better than SAT for 3D.
- It uses the Minkowski Difference ($A \ominus B$) instead of projections:
  - Take every point in Shape A and subtract every point in Shape B.
  - If the two shapes are overlapping, their Minkowski difference will contain the origin $(0,0,0)$ $\Rightarrow$There is a point P_A in A and a point P_B in B such that $P_A = P_B$!



The distance between A and B is equivalent to the distance between their Minkowski difference and the origin

Narrow Phase: Exact Computations (GJK)

- Iterative process to avoid computing the whole shape.
- **Support** function: Pick a direction $\mathbf{d}$ and find the point in the Minkowski shape furthest along that direction.
- GJK builds a simplex (point→line→triangle→tetrahedron→…) inside the Minkowski shape.
- Goal: Iteratively move the simplex towards the origin.
  - If it encloses the origin: **collision detected**
  - If it doesn't: **no collision**



[Christer Ericsson's Real-Time Collision Detection]

Collision Response

- In a discrete simulation, collisions happen “instantly” between two frames. If we used a standard force, we would need a massive force and a tiny time step to stop an object.

- Inpulse ($\mathbf{J}$) is the integral of force over time. It causes an instantaneous change in velocity:
  $\mathbf{J} = \Delta\mathbf{p} = m(\mathbf{v}_{new} - \mathbf{v}_{old})$

- Coefficient of restitution (e): It defines the “bounciness” of the collision:
  - $e = 1$ for perfectly elastic (billiard balls)
  - $e = 0$ for perfectly inelastic (lump or clay)

- Newton’s law of restitution:
  $(\mathbf{v}'_a - \mathbf{v}'_b) \cdot \mathbf{n} = -e((\mathbf{v}_a - \mathbf{v}_b) \cdot \mathbf{n})$

- Impulse equation:

$$j = \frac{-(1+e)(\mathbf{v}_{rel} \cdot \mathbf{n})}{\frac{1}{m_a} + \frac{1}{m_b} + [(\mathbf{r}_a \times \mathbf{n})^T I_a^{-1}(\mathbf{r}_a \times \mathbf{n})] + [(\mathbf{r}_b \times \mathbf{n})^T I_b^{-1}(\mathbf{r}_b \times \mathbf{n})]}$$

- Key terms:
  - $\mathbf{n}$: collision normal from GJK/SAT
  - $\mathbf{r}$: vector from center of mass to contact point
  - I^{-1}: inverse Inertia Tensor (how easy it spins)

- Applying the result (where $\mathbf{J} = j\mathbf{n}$):
  - Linear velocity update: $\mathbf{v}_{new} = \mathbf{v}_{old} + \frac{\mathbf{J}}{m}$
  - Angular velocity update: $\omega_{new} = \omega_{old} + I^{-1}(\mathbf{r} \times \mathbf{J})$

Summary & Real-world Trade-offs

- Accuracy vs Stability vs Performance
- The Future: Machine learning-based physics and GPU-accelerated solvers